10 - TCL: TRANSAÇÕES E INTEGRIDADE REFERENCIAL

Banco de Dados • COMMIT • ROLLBACK • SAVEPOINT • ACID • Foreign Keys • Projeto SmartCoffee

Abertura Inspiradora

> **“Uma operação só deve ser considerada concluída quando todas as partes necessárias foram concluídas com sucesso.”**

Apresentação

Na Aula 08 consultamos dados com DQL. Na Aula 09 manipulamos dados com `INSERT`, `UPDATE` e `DELETE`. Agora surge uma pergunta essencial:

O que acontece quando uma operação possui várias etapas e uma delas falha no meio do processo?

É nesse momento que entram as **transações**. Elas permitem tratar várias operações como uma unidade lógica de trabalho.

Objetivos da Aula

- compreender o conceito de transação;
- usar `START TRANSACTION`, `COMMIT`, `ROLLBACK` e `SAVEPOINT`;
- compreender `AUTOCOMMIT`;
- conhecer ACID em nível introdutório;
- relacionar transações com DML e constraints;
- interpretar `ON DELETE` e `ON UPDATE`;
- executar operações seguras no SmartCoffee.

---

1. O problema que a transação resolve

Uma venda pode envolver criar PEDIDO, inserir ITEM_PEDIDO, registrar PAGAMENTO e atualizar ESTOQUE. Se uma etapa falhar, deixar apenas parte do processo gravada pode gerar inconsistência.

Sem transação

Parte da venda pode permanecer gravada.

Com transação

Podemos confirmar tudo ou desfazer tudo.

---

2. Conceito de transação

Transação é um conjunto de operações tratado como uma única unidade lógica.

```
START TRANSACTION;

INSERT ...
UPDATE ...
INSERT ...

COMMIT;
```

INÍCIO → OPERAÇÕES → VALIDAÇÃO → COMMIT ou ROLLBACK

---

3. COMMIT

`COMMIT` confirma as alterações realizadas dentro da transação.

```
START TRANSACTION;

UPDATE produto
SET preco = 8.50
WHERE id_produto = 1;

COMMIT;
```

Após o COMMIT, um ROLLBACK posterior não desfaz aquela transação já confirmada.

---

4. ROLLBACK

`ROLLBACK` desfaz as alterações ainda não confirmadas da transação atual.

```
START TRANSACTION;

UPDATE produto
SET preco = 5000.00
WHERE id_produto = 1;

SELECT * FROM produto WHERE id_produto = 1;

ROLLBACK;
```

---

5. SAVEPOINT

Um **SAVEPOINT** cria um ponto intermediário dentro de uma transação.

```
START TRANSACTION;

UPDATE produto
SET preco = preco * 1.05
WHERE id_categoria = 1;

SAVEPOINT reajuste_cafes;

UPDATE produto
SET preco = preco * 1.50
WHERE id_categoria = 4;

ROLLBACK TO SAVEPOINT reajuste_cafes;

COMMIT;
```

Analogia: SAVEPOINT funciona como um checkpoint.

---

6. RELEASE SAVEPOINT

```
RELEASE SAVEPOINT reajuste_cafes;
```

Remove o savepoint da transação atual, mas não executa COMMIT.

---

7. AUTOCOMMIT

O MySQL normalmente trabalha com autocommit ativo fora de transações explícitas.

```
SELECT @@autocommit;
```

Para um processo composto por várias operações relacionadas, prefira:

```
START TRANSACTION;
-- DMLs relacionadas
COMMIT;
```

---

8. Onde TCL se encaixa?

Grupo	Finalidade	Comandos
DDL	estrutura	CREATE, ALTER, DROP
DQL	consulta	SELECT
DML	manipulação	INSERT, UPDATE, DELETE
TCL	**controle de transações**	**COMMIT, ROLLBACK, SAVEPOINT**

---

9. ACID

Propriedade	Ideia	Exemplo SmartCoffee
Atomicidade	tudo ou nada	pedido e seus itens não ficam pela metade

Propriedade	Ideia	Exemplo SmartCoffee
Consistência	regras continuam válidas	FK, CHECK, UNIQUE e NOT NULL são respeitados
Isolamento	transações concorrentes não interferem indevidamente	dois caixas podem operar ao mesmo tempo
Durabilidade	COMMIT torna o resultado persistente	pedido confirmado permanece armazenado

Nesta aula, o foco prático é principalmente **atomicidade e consistência**.

---

10. Transação não substitui CONSTRAINT

Transação

Controla se um conjunto de operações será confirmado ou desfeito.

Constraint

Impede que determinados estados inválidos sejam aceitos.

```
CONSTRAINT ck_estoque_quantidade
CHECK (quantidade >= 0)
```

Se um UPDATE tentar deixar o estoque negativo, a constraint bloqueia a operação; o fluxo pode então executar ROLLBACK.

---

11. Integridade referencial

```
CONSTRAINT fk_pedido_cliente
FOREIGN KEY (id_cliente)
REFERENCES cliente(id_cliente)
```

A FK impede que PEDIDO aponte para um CLIENTE inexistente.

---

12. ON DELETE

Ação	Comportamento	Uso possível
RESTRICT / NO ACTION	bloqueia exclusão com dependentes	preservar histórico
CASCADE	exclui dependentes	dependente não faz sentido sem o pai
SET NULL	remove a referência	relação opcional e FK aceita NULL

```
CONSTRAINT fk_item_pedido
FOREIGN KEY (id_pedido)
REFERENCES pedido(id_pedido)
ON DELETE CASCADE;
```

CASCADE não é automaticamente a melhor escolha. Excluir CLIENTE e todos os PEDIDOS pode destruir histórico de vendas.

---

13. ON UPDATE

```
ON UPDATE CASCADE
```

Define o que ocorre com FKs quando a chave referenciada é alterada. IDs substitutos normalmente não deveriam mudar sem motivo, mas é importante conhecer o mecanismo.

---

14. Exclusão física × lógica

Física

```
DELETE FROM produto
WHERE id_produto = 10;
```

Lógica

```
UPDATE produto
SET ativo = FALSE
WHERE id_produto = 10;
```

Para produtos, clientes ou outros registros com histórico, a exclusão lógica pode ser mais adequada.

---

15. Fluxo seguro para UPDATE/DELETE

```
SELECT * FROM produto WHERE id_produto = 3;

START TRANSACTION;

UPDATE produto
SET preco = 12.50
WHERE id_produto = 3;

SELECT * FROM produto WHERE id_produto = 3;

COMMIT;
-- ou ROLLBACK;
```

---

16. Demonstração: pedido completo

```
START TRANSACTION;

INSERT INTO pedido (id_cliente, status_pedido)
VALUES (1, 'ABERTO');
```

```
SET @pedido = LAST_INSERT_ID();

INSERT INTO item_pedido
(id_pedido, id_produto, quantidade, preco_unitario)
VALUES
(@pedido, 1, 2, 7.50),
(@pedido, 5, 1, 8.00);

INSERT INTO pagamento
(id_pedido, valor, forma_pagamento)
VALUES (@pedido, 23.00, 'PIX');

COMMIT;
```

---

17. Demonstração: erro no meio do processo

```
START TRANSACTION;

INSERT INTO pedido (id_cliente, status_pedido)
VALUES (1, 'ABERTO');

SET @pedido = LAST_INSERT_ID();

-- produto inexistente → erro de FK
INSERT INTO item_pedido
(id_pedido, id_produto, quantidade, preco_unitario)
VALUES (@pedido, 9999, 1, 10.00);

ROLLBACK;
```

O objetivo é não deixar o pedido parcial gravado.

---

18. Transação envolvendo estoque

```
START TRANSACTION;

INSERT INTO pedido (...) VALUES (...);
SET @pedido = LAST_INSERT_ID();

INSERT INTO item_pedido (...) VALUES (...);
```

```
UPDATE estoque
SET quantidade = quantidade - 2
WHERE id_produto = 1;

INSERT INTO pagamento (...) VALUES (...);

COMMIT;
```

Se qualquer etapa falhar, o processo pode ser desfeito antes da confirmação.

---

19. DDL e transações

Para esta aula, utilize transações principalmente com **DML**. Comandos DDL podem provocar comportamentos de confirmação próprios do SGBD.

Regra didática: os exercícios de COMMIT e ROLLBACK usarão INSERT, UPDATE e DELETE.

---

20. Erros comuns

- executar COMMIT antes de conferir;
- achar que ROLLBACK desfaz algo já confirmado;
- esquecer uma transação aberta;
- usar CASCADE sem analisar impacto;
- confundir transação com integridade referencial;
- executar UPDATE/DELETE sem validar o WHERE;
- permitir processo parcialmente gravado.

---

21. Questões para discussão

1. Uma venda deve ser confirmada se o pagamento falhar?
2. Excluir um cliente deve excluir seus pedidos?
3. Excluir um pedido deve excluir seus itens?
4. Produto fora do cardápio deveria ser excluído ou inativado?
5. Qual é o melhor momento para executar COMMIT?
6. Quando um SAVEPOINT é útil?

---

22. Atividade Prática — Operação Segura SmartCoffee

Missão: executar operações reais, provocar erros propositalmente e preservar a consistência utilizando constraints e transações.

Parte A — ROLLBACK

1. consulte o preço de um produto;
2. inicie transação;
3. altere o preço;
4. consulte novamente;
5. execute ROLLBACK;
6. comprove o retorno do valor anterior.

Parte B — COMMIT

1. repita a alteração;
2. confira;
3. execute COMMIT;
4. comprove a persistência.

Parte C — SAVEPOINT

1. faça uma alteração;
2. crie um SAVEPOINT;
3. faça outra;
4. volte ao SAVEPOINT;
5. COMMIT;
6. explique o resultado.

Parte D — Venda completa

Em uma única transação, cadastre um pedido, dois itens e um pagamento. Consulte tudo antes do COMMIT.

Parte E — Falha proposital

Repita a venda usando um produto inexistente. Observe a FK e execute ROLLBACK.

Parte F — Integridade referencial

- tente excluir categoria com produtos;
- observe ON DELETE CASCADE em pedido e itens;
- tente usar FK inexistente;

- inative um produto em vez de excluí-lo.

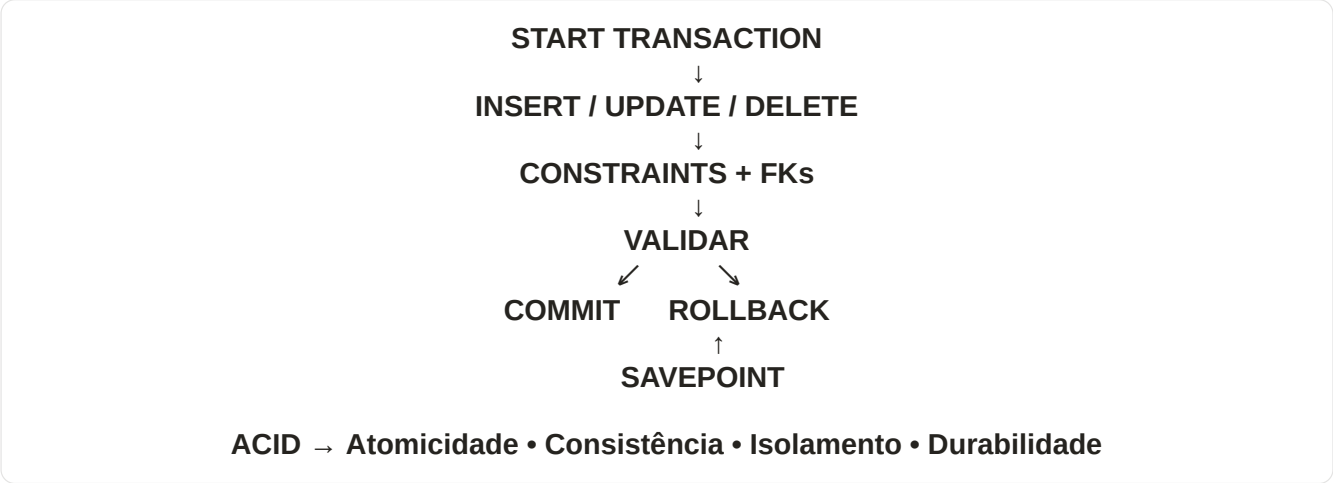
---

23. Desafio Final — Caixa do SmartCoffee

1. START TRANSACTION;
2. criar PEDIDO;
3. armazenar o ID;
4. inserir 3 itens;
5. atualizar o ESTOQUE;
6. criar SAVEPOINT;
7. registrar PAGAMENTO;
8. alterar status para FINALIZADO;
9. consultar tudo;
10. COMMIT se estiver correto ou ROLLBACK se houver erro.

---

24. Mapa Mental



Encerramento

> **Uma transação transforma várias instruções SQL em uma única decisão de negócio: confirmar tudo ou não confirmar nada.**